NAME:- Shubham Biradar Biradar

ROLL NO. :- A-12

BATCH:-A1

ASSINGMENT:- 4

Aim:- Numerical Computation with NumPy: Perform different computations using NumPy library.

Q.1. Explain the role of NumPy in the Python scientific computing ecosystem. Answer:- NumPy (Numerical Python) plays a fundamental role in the Python scientific computing ecosystem by providing efficient tools for numerical calculations and data manipulation. Its core feature is the ndarray (n-dimensional array), which allows fast storage and operations on large datasets compared to regular Python lists. NumPy supports vectorized operations, meaning calculations can be performed on entire arrays without using loops, making computations faster and more efficient. It also includes a wide range of mathematical, statistical, and linear algebra functions, which are essential for scientific analysis. Additionally, NumPy serves as the foundation for many other libraries such as Pandas, SciPy, and Matplotlib, enabling seamless integration and data sharing between them. Overall, NumPy enhances performance, simplifies complex calculations, and acts as a backbone for scientific and data-driven applications in Python.
Q.2. Why is NumPy considered faster than pure Python computations? Answer:- NumPy is considered faster than pure Python computations mainly because it is implemented in **C language**, which allows it to execute operations at a much lower level and with higher efficiency than standard Python code. Unlike Python lists, NumPy arrays store data in **contiguous memory blocks**, improving cache performance and reducing memory overhead. Another key reason is **vectorization**, where operations are applied to entire arrays at once instead of using slow Python loops, significantly speeding up computation. NumPy also uses **optimized mathematical libraries** (like BLAS and LAPACK) for complex calculations such as linear algebra. Additionally, it reduces the need for dynamic type checking because all elements in a NumPy array have the same data type, which further improves execution speed. Together, these features make NumPy much faster and more efficient for numerical and scientific computing tasks.

Q.3. What types of mathematical operations can be performed using NumPy? Answer:- NumPy supports a wide variety of mathematical operations that are essential for scientific and numerical computing. It allows performing **basic arithmetic operations** such as addition, subtraction, multiplication, and division on arrays in an element-wise manner. It also provides **statistical functions** like mean, median, standard deviation, variance, minimum, and maximum, which are useful for data analysis. NumPy includes powerful **linear algebra operations** such as matrix multiplication, determinant, inverse, eigenvalues, and solving linear equations. In addition, it supports **trigonometric functions** like sine, cosine, and tangent, as well as **logarithmic and exponential functions**. NumPy can also handle **rounding operations** (round, floor, ceil) and **aggregation functions** like sum and product. Overall, it offers a comprehensive set of mathematical tools that make complex calculations efficient and easy to perform.
Q.4.Explain indexing and slicing in NumPy arrays **Answer:-**Indexing and slicing in NumPy arrays are techniques used to access and manipulate specific elements or subsets of data efficiently. Indexing refers to accessing individual elements of an array using their position (index). In NumPy, indexing starts from 0, so the first element is accessed with index 0. You can also use negative indexing to access elements from the end (e.g., -1 for the last element). In multi-dimensional arrays, multiple indices are used to access elements (e.g., row and column positions in a 2D array).

Slicing, on the other hand, is used to extract a portion (subarray) of the array. It is done using the syntax start:stop:step, where start is the beginning index, stop is the ending index (not included), and step defines the interval. For example, arr[1:5] selects elements from index 1 to 4. In multi-dimensional arrays, slicing can be applied across rows and columns simultaneously (e.g., selecting specific rows and columns from a matrix).
Q.5.How does NumPy integrate with libraries like: a. Pandas b. Matplotlib c. Scikit-learn Answer:- NumPy integrates closely with other Python libraries and acts as a foundation for efficient data handling and computation.
a. Pandas: Pandas is built directly on top of NumPy. It uses NumPy arrays as the underlying data structure for its Series and DataFrame objects. This allows Pandas to perform fast data manipulation and analysis. Many operations in Pandas, such as filtering, aggregation, and statistical analysis, rely on NumPy functions, ensuring high performance and efficient memory usage.

b. Matplotlib: Matplotlib uses NumPy arrays to handle numerical data for visualization. When creating graphs like line plots, bar charts, or histograms, the data is often passed as NumPy arrays. NumPy makes it easy to generate and manipulate datasets (e.g., using linspace, arange), which can then be directly plotted using Matplotlib, enabling smooth and efficient visualization of large datasets.

c. Scikit-learn: Scikit-learn relies heavily on NumPy arrays for machine learning tasks. Input data (features and labels) is typically stored as NumPy arrays, which are then used for training and testing models. NumPy's fast computations and support for matrix operations make it ideal for algorithms like regression, classification, and clustering. This integration ensures that machine learning models run efficiently and handle large datasets effectively.

```
#Perform basic arithmetic operations on arrays.
import numpy as np
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
add = a + b
sub = a - b
mul = a * b
div = a / b
print("Addition:", add)
print("Subtraction:",                      sub)
print("Multiplication:",                   mul)
print("Division:", div)
```

```
Addition: [11 22 33]
Subtraction: [ 9 18 27]
Multiplication: [10 40 90]
Division: [10. 10. 10.]
```

```
#Use NumPy functions for mathematical operations.
import numpy as np
arr = np.array([4, 9, 16, 25])
sqrt_val = np.sqrt(arr)
exp_val = np.exp(arr)
log_val = np.log(arr)
sin_val = np.sin(arr)
cos_val = np.cos(arr)
sum_val = np.sum(arr)
mean_val = np.mean(arr)


print("Square root:", sqrt_val)
print("Exponential:", exp_val)
print("Logarithm:", log_val)
print("Sine:", sin_val)
print("Cosine:", cos_val)
print("Sum:", sum_val)
print("Mean:", mean_val)
```

```
Square root: [2. 3. 4. 5.]
Exponential: [5.45981500e+01 8.10308393e+03 8.88611052e+06 7.20048993e+10]
Logarithm: [1.38629436 2.19722458 2.77258872 3.21887582]
Sine: [-0.7568025   0.41211849 -0.28790332 -0.13235175]
Cosine: [-0.65364362 -0.91113026 -0.95765948 0.99120281]
Sum: 54
Mean: 13.5
```

```
#Implement vectorized operations for performance comparison.
import numpy as np
import time
size = 1000000
a = np.random.rand(size)
b = np.random.rand(size)


start = time.time()
result_loop = []
for i in range(size):
    result_loop.append(a[i] + b[i])
end = time.time()
print("Loop Time:", end - start)

start = time.time()
result_vector = a + b
end = time.time()
print("Vectorized Time:", end - start)
```

```
Loop Time: 0.43634533882141113
Vectorized Time: 0.004973649978637695
```

```
#Optimize numerical computations using NumPy functions.
import numpy as np
import time
arr = np.random.rand(1000000)
start = time.time()
total = 0
for i in arr:
 total += i**2
end = time.time()
print("Loop Time:", end - start)
start = time.time()
total_np = np.sum(arr**2)
end = time.time()
print("NumPy Time:", end - start)
```

```
Loop Time: 0.6364316940307617
NumPy Time: 0.0058727264404296875
```

Conclusion:-In conclusion, numerical computation with NumPy plays a vital role in performing fast and efficient mathematical operations in Python. By using NumPy's powerful array structure and built-in functions, complex calculations such as arithmetic operations, linear algebra, statistical analysis, and random number generation can be executed and accurately. It reduces the need for manual loops through vectorization, improving performance and saving time. Overall, NumPy provides a strong foundation for scientific computing and makes numerical data processing more efficient, reliable, and scalable.